San Jose State University
CS 147 - Computer Architecture
Class Project - Phase 3

Overall Timeline and Grade Distribution

Due Date	Project Component	
2/26/26	Design Review	(5% of project grade)
3/10/26	Phase #1	(15% of project grade)
4/7/26	Phase #2	(30% of project grade)
4/16/26	Phase #2.1	(10% of project grade)
N/A	Phase #2.2	(Optional)
4/23/26	Phase #2.3 Caching	(10% of project grade)
5/7/26	Phase #3	(30% of project grade)

Phase #3 — Final Processor Integration (30% of project grade)

Overview

Phase 3 is the final stage of the processor project. In this phase you will integrate your fully pipelined processor with a realistic memory system and implement several required performance optimizations.

By this point, your design should already include:

- A working five-stage pipelined processor
- A functioning cache system
- Correct handling of pipeline hazards

Phase 3 focuses on integrating the full memory hierarchy and implementing the forwarding and prediction mechanisms needed for efficient execution.

Required Features

Your final processor must implement the following functionality:

- Two-way set-associative caches backed by multi-cycle memory
- Register file bypassing
- Forwarding from the beginning of the MEM stage to the beginning of the EX stage
- Forwarding from the beginning of the WB stage to the beginning of the EX stage
- Static branch prediction (predict not taken)

When implementing forwarding, ensure that your design does not create a combinational path that effectively performs the work of multiple pipeline stages in a single cycle. Violating the intended pipeline structure may result in loss of credit.

The IPC of your processor will also account for a small portion of this demo grade, so you should attempt to eliminate unnecessary stall cycles where possible.

Assembling / Simulating

For the WISC-SJ26 ISA specification, refer to:

```
./assignments/project/common/writeup/wisc-sj26_spec.pdf
```

An assembler for the WISC-SJ26 ISA is provided for your use. Sample test programs are also provided, although you are strongly encouraged to write custom tests to augment these. Be aware that some provided test programs were written for a slightly different ISA specification and therefore may not work exactly as advertised. If unexpected behavior occurs, you must determine whether the issue is in your design or in the assumptions made by the test itself.

The expected workflow for debugging WISC-SJ26 assembly tests is:

1. Write a test in WISC-SJ26 assembly language.
2. Assemble it using:

   ```
   ./run assemble <program>.asm <program>
   ```

3. Simulate the generated image using:

   ```
   ./run simulate <program>_all.img
   ```

4. Examine the simulator trace to confirm that each instruction and register update matches your expectations.

The simulator prints an instruction-by-instruction execution trace, which should be used to confirm that your program is doing exactly what you intended before you rely on it as evidence that the hardware is correct.

Cache Files To Carry Forward

Your final cache implementation work for this project path should live in:

```
./demo2/verilog/phase2_3/cache_assoc/
```

For Phase 3, the carried-forward cache should be the **two-way set-associative** version that serves as the final cache design target.

The carried-forward cache files should include:

File	Description
cache.v	Cache storage structures (data, tag, valid, dirty)
clkrst.v	Clock and reset module
final_memory.v	Memory bank implementation
four_bank_mem.v	Four-banked memory wrapper
mem.addr	Example address trace file
memc.v	Data storage module used by cache
memv.v	Valid-bit storage module used by cache
mem_system_hier.v	Top-level wrapper used for testing
mem_system_perfbench.v	Trace-based testbench
mem_system_randbench.v	Random testbench
mem_system_ref.v	Reference implementation used by testbench
mem_system.v	Memory system module **(this is the file you modify)**
.syn.v	Synthesizable memory modules

Place the carried-forward cache implementation in:

```
./demo3/verilog/phase2_3/cache_assoc/
```

Only the associative/final implementation should be copied forward for Phase 3.

Your cache controller state diagram should also be carried forward.

Required student_custom tests

Place your custom assembly tests in:

```
./assignments/project/testprograms/student_custom/
```

and add them to:

```
./assignments/project/testprograms/student_custom/all_phase_3.list
```

At least two custom tests are required. Writing more tests is strongly encouraged.

Synthesis

You should also synthesize your processor and submit the results of synthesis, including the area, cell, and timing reports. This is similar to what you did for Lab 3.

Submission Contents

verilog/ This directory should contain all Verilog and Vcheck files needed to compile and run your processor. Your processor should be able to compile and run using only the files present in this directory.

The Phase 3 testing and submission flow is intended to match the rest of the course workflow: you should run the project test command, then generate your submission using the standard submission wrapper. Any result collection and packaging details should be handled automatically by the submission process.

Running the full verification suite may take approximately 40 minutes, so plan accordingly.

Grading and Demonstration

Submissions will ultimately be graded automatically using the course verification flow for Phase 3.

If failures are detected, students may be asked to meet with the course staff to discuss partial credit and debugging.

If your processor has known failures, you should bring a short written explanation of the causes of those failures. Demonstrating an understanding of the problem will help maximize the credit you receive.

If the complete processor is not functional, you may demonstrate partially working subsystems such as:

- Stalling instruction memory alone
- Stalling data memory alone
- Stalling instruction and data memory together
- Direct-mapped instruction cache
- Direct-mapped data cache
- Two-way instruction cache
- Two-way data cache

During any demonstration, you should be prepared to explain the behavior of the processor design.

Testing

Use:

```
./run test [-v] project phase_3 [subtest]
```

Submitting

Use:

```
./run build_submission project phase_3
```